EE5203 Lab 3 Guide

Functions and registers: a sensor module and an LED driver for LD2 - Basri Erdogan

Mission: Move your Lab 2 calculations into C functions, and write a small LED driver that controls LD2 (pin PA5) through CMSIS registers. At the end, LD2 blinks the status of your eight sensor values: **1 blink = LOW, 2 blinks = OK, 3 blinks = HIGH**.

This lab uses Lecture 3 (functions, header files, `CMakeLists.txt`) and Lecture 4 (registers, bit masks, `->`).

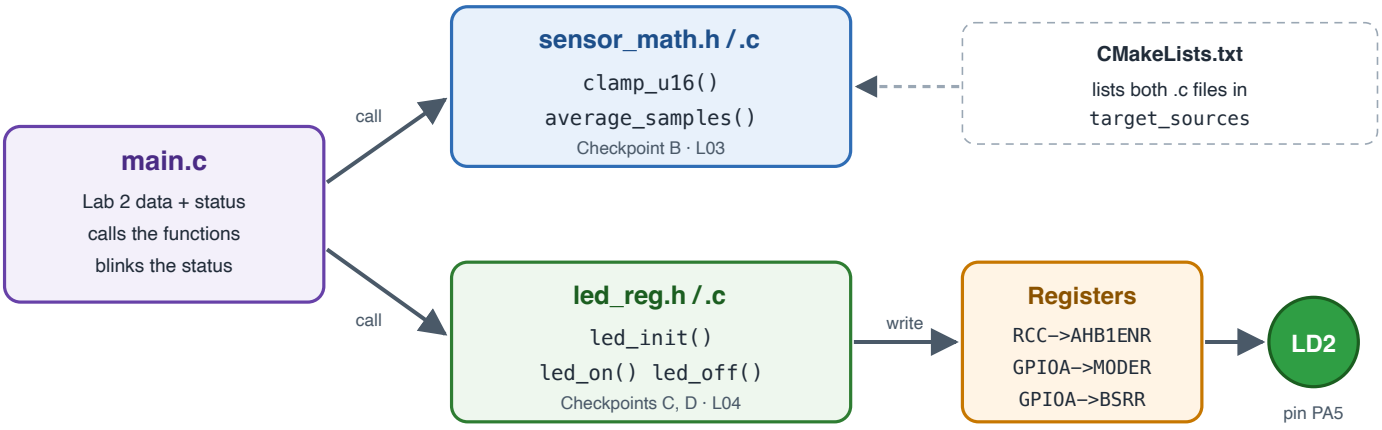


Figure 1: What you build: `main.c` calls two small modules; the LED module writes three registers that drive LD2.

Prepare before class

- ☐ Bring your board and a USB data cable, and your **Lab 2 results** (the Checkpoint D table). Lab 3 must give the same values.
- ☐ Download the starter files: `EE5203_L03_starter.zip`.
- ☐ Do the **Lab 3 pre-lab quiz** on CATS. It includes the bit-mask table below. To practise masks first, use the Bitwise Lab.
- ☐ Bring this table, completed on paper:

Purpose	Expression	Hex value
GPIOA clock bit	`1U << 0`	
PA5 mode field (two bits)	`3U << 10`	
PA5 output pattern `01`	`1U << 10`	
Set PA5	`1U << 5`	
Reset PA5	`1U << 21`	

Learning objectives

By checkoff time, you can:

1. write a function declaration in a `.h` file and its definition in a `.c` file;
2. add a new `.c` file to the build in `CMakeLists.txt`;
3. replace repeated code in `main.c` with function calls and get the same results;
4. set and clear register bits with `|=`, `&= ~` and masks;
5. show in the debugger that a register value agrees with what LD2 does.

Hardware and files

Item	Value
Board	Nucleo-F401RE
Tools	STM32CubeMX (new project) + VS Code
Starting point	A new CubeMX project; the Lab 2 code is given in Checkpoint B
Project name	`EE5203_L03_<studentnumber>`
New files (starter)	`sensor_math.h`, `sensor_math.c`, `led_reg.h`, `led_reg.c`
Files you edit	`CMakeLists.txt`, `main.c`, and the four new files
Registers	`RCC->AHB1ENR`, `GPIOA->MODER`, `GPIOA->BSRR`

Keep all edits to `main.c` inside the `USER CODE` regions.

Checkpoint A - Create a new project and add the starter files

Start of the session: if `C:\Work\<studentnumber>` does not contain your Lab 2 project, copy **all** your project folders from `H:\EE_Labs` into it first (as in Lab 2). Do not run *Save to H* now.

You make a **new** project. Do not change your Lab 2 project.

1. In **STM32CubeMX**: Board Selector → **NUCLEO-F401RE** → accept the default initialization (as in Lab 1). PA5 is already LD2.
2. Project Manager: name `EE5203_L03_<studentnumber>`, location `C:\Work\<studentnumber>`, **Toolchain/IDE = CMake** → Generate Code.
3. Copy the folder `C:\Users\Public\EE5203_template\.vscode` into the new project folder, next to the `.ioc` file.
4. Open `EE5203_L03_starter.zip`. Copy the `Core` folder inside it and paste it into your project folder. Two files go to `Core\Inc`, two to `Core\Src`.
5. In `CMakeLists.txt`, add the two new `.c` files to `target_sources`, below the line `# Add user sources here`. Write the paths yourself, in the same form as `Core/Src/<file>.c`. (The project name is already correct: CubeMX used your folder name.)
6. Open the folder in VS Code and build with **Ctrl+Shift+B**: zero errors. The starter files build even though they are not finished. Two warnings (`unused parameter 'low'` and `'high'`) are normal: they go away when you finish `clamp_u16` in Checkpoint B.

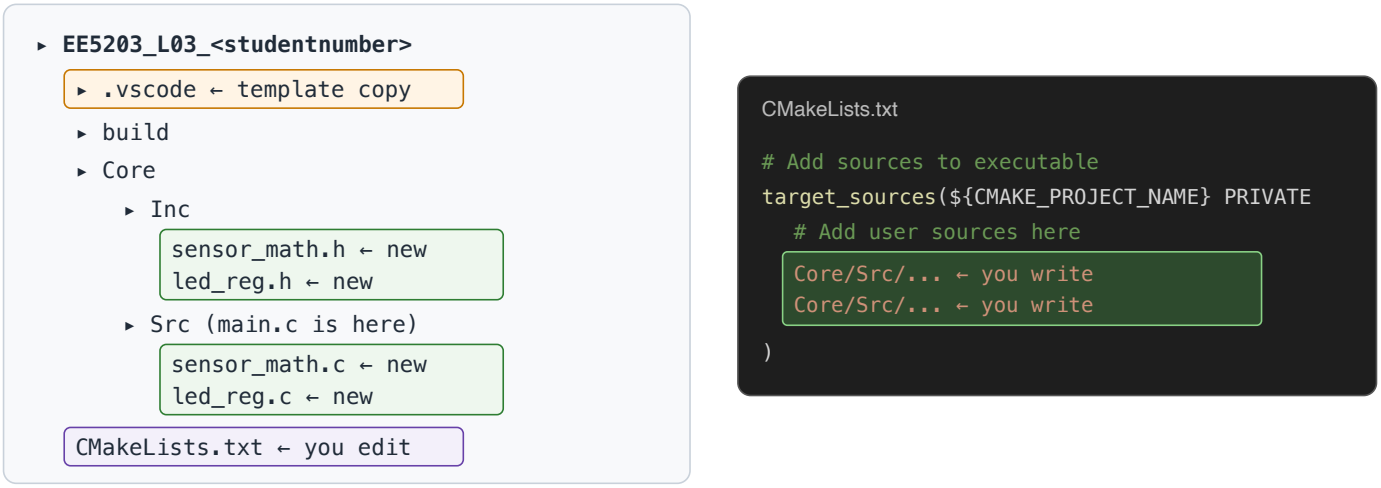


Figure 2: Your project folder after Checkpoint A.

Show: the four new files in the Explorer, your two lines in `CMakeLists.txt`, and the build line `Linking C executable EE5203_L03_<studentnumber>.elf`.

Checkpoint B - Move the Lab 2 calculations into functions

You write two short functions, then call them from `main.c`.

B1 - Write the two functions

1. In `sensor_math.c`, complete the **body** of `clamp_u16`: return `low` if the value is below `low`, `high` if it is above `high`, and the value itself otherwise.
2. In `sensor_math.h`, replace the `TODO` comment for `average_samples` with this **declaration**:

```
uint16_t average_samples(uint16_t values[], int count);
```

3. In `sensor_math.c`, replace the `TODO` comment for `average_samples` with this **definition**, and write the loop yourself:

```
uint16_t average_samples(uint16_t values[], int count)
{
    uint32_t sum = 0;
    /* Your loop: add every values[i] to sum. Use i < count. */

    return sum / count;
}
```

4. Build: zero errors.

B2 - Call the functions from `main.c`

1. Add `#include "sensor_math.h"` inside `USER CODE BEGIN Includes`.

Add steps 2-4 inside `USER CODE BEGIN 2`, one below the other.

2. The eight values from Lab 2:

```
uint16_t samples[8] = {850, 1250, 4095, 4200, 2050, 900, 3100, 2800};
```

3. Clamp every value with your function:

```
for (int i = 0; i < 8; i++) {
    samples[i] = clamp_u16(samples[i], 0, 4095);
}
```

4. Calculate the average with your function:

```
uint16_t average = average_samples(samples, 8);
```

5. Build and start the debugger.

The results must be **the same as in Lab 2**. Copy the Lab 2 values from your Lab 2 table, then read the Lab 3 values in the debugger:

Quantity	Lab 2 value	Lab 3 value (debugger)
`samples[3]` after the loop		
`average`		

Show: a breakpoint inside the loop of `average_samples`, with `i` and the sum changing as you step, and the completed table.

Checkpoint C - Write `led_init()` and check the registers

`led_reg.h` and `led_reg.c` contain only comments. You write the code. The three registers you use in Checkpoints C and D:

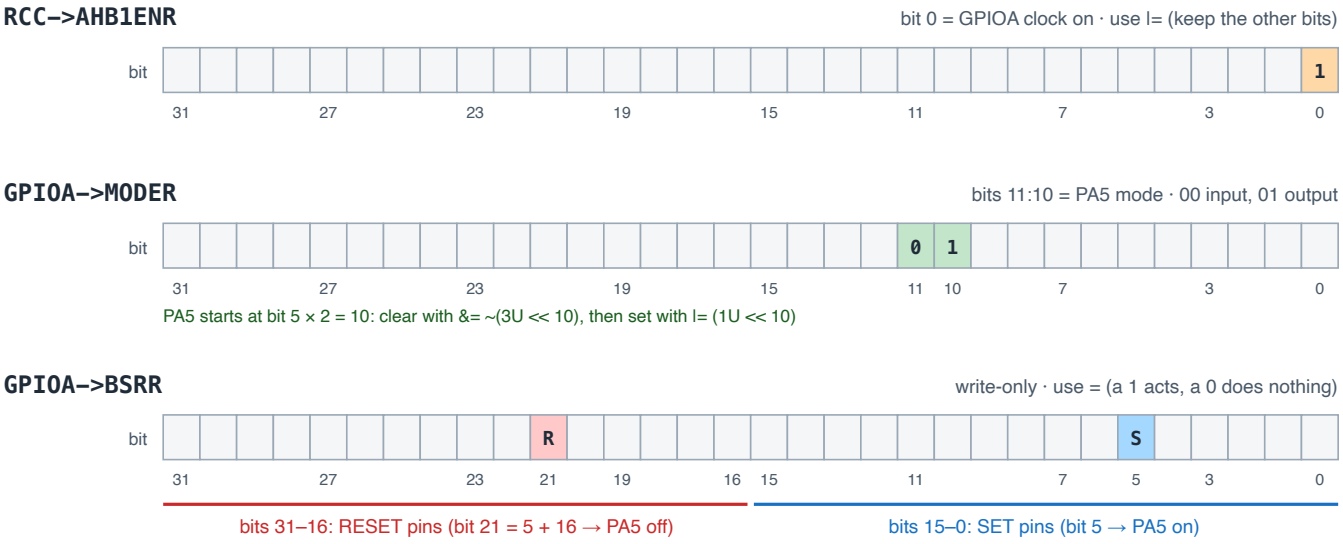


Figure 3: The bits you change: AHB1ENR bit 0, MODER bits 11:10 for PA5, BSRR bit 5 (set) and bit 21 (reset).

1. In `led_reg.h`, **declare** three functions with no parameters and no return value: `led_init`, `led_on`, `led_off`.
2. In `led_reg.c`, **define** `led_init` with three statements:
   - set bit 0 of `RCC->AHB1ENR` (GPIOA clock) without changing other bits;
   - clear bits 11:10 of `GPIOA->MODER`;
   - write the pattern `01` into bits 11:10 (PA5 = output).
3. In `main.c`, add `#include "led_reg.h"` inside `USER CODE BEGIN Includes`.
4. Call `led_init();` inside `USER CODE BEGIN SysInit`. This region is just **above** `MX_GPIO_Init();`, so your function runs **before** the CubeMX code:

```
/* USER CODE BEGIN SysInit */
led_init();
/* USER CODE END SysInit */
```

At this point nothing has touched GPIOA yet: the clock bit is `0` and PA5 is in input mode (`00`). Your three statements change both.

Break on `led_init();`, then step **into** it (F11) and over each statement. Look at GPIOA and RCC in the register (SFR) view:

Moment	`RCC->AHB1ENR` bit 0	`MODER` bits 11:10
Before `led_init`		
After the clock line		
After the clear		
After the set		

Then step over `MX_GPIO_Init();`. CubeMX writes the same values, so nothing changes.

Show: the table, and answer: if the clock bit is `0`, what happens to a write to `GPIOA->MODER`?

Checkpoint D - Turn LD2 on and off with `BSRR`

1. In `led_reg.c`, **define** `led_on` and `led_off`. Each is one assignment to `GPIOA->BSRR`: bit 5 sets PA5, bit 21 (5 + 16) resets it.
2. In `main.c`, inside `USER CODE BEGIN 3`, add this blink (LD2 on, wait, off, wait; the `while (1)` loop repeats it):

```
led_on();
HAL_Delay(200);
led_off();
HAL_Delay(200);
```

   Do not use any `HAL_GPIO_...` call.
3. Put a breakpoint on `led_off();`. When it stops, LD2 must be **on**. Step over: LD2 turns **off**.

Show: LD2 blinking, your three functions, and the register view when the program stops at `led_off();` (`ODR` bit 5 = 1).

Checkpoint E (optional, not graded) - Blink the status

First add the Lab 2 status rule in `USER CODE BEGIN 2`, below the `average` line:

```
int status_code = 0;
if (average < 1500) {
    status_code = -1;      /* LOW  */
} else if (average > 3000) {
    status_code = 1;       /* HIGH */
}                          /* otherwise 0 = OK */
```



Figure 4: The three blink patterns.

1. Calculate a blink count: LOW = 1, OK = 2, HIGH = 3. One short expression is enough.
2. In `USER CODE BEGIN 3`, replace the Checkpoint D blink: blink LD2 that many times (200 ms on, 200 ms off), then wait 1000 ms. Use a `for` loop.
3. Predict the number of blinks for your data before you run it.

Show: the blink pattern, your prediction, and the code.

If you finish early: change one of your samples so that the status becomes HIGH, predict the new blink count, and check it.

Common problems

What you see	First check
`undefined reference to 'average_samples'` or `'led_init'`	The `.c` file is in `target_sources`. The name in the `.c` file matches the `.h` file.
`implicit declaration of function 'led_on'`	`#include "led_reg.h"` is in `main.c`, and the declaration is in the header.
`'GPIOA' undeclared` in `led_reg.c`	`#include "main.h"` is at the top of `led_reg.c`.
`conflicting types for 'average_samples'`	The declaration and the definition use exactly the same types.
LD2 never lights after `led_init`	Check the `MODER` bits in the register view: they must be `01`.
`MODER` stays `00` while you step through `led_init`	The clock line must come **first**. Without the clock, writes to GPIOA are lost.
LD2 stays on	`led_off` uses bit 21, not bit 5.
The average differs from Lab 2	Stop inside `average_samples`; watch `i`, `count` and the sum.
`...FLASH.ld: syntax error`	Build with **Ctrl+Shift+B** (the template repairs the CubeMX linker script). Check that `.vscode` is in the project folder.

End of the session - Save to H

Close VS Code, run the **Save to H** desktop shortcut, and check that `H:\EE_Labs\EE5203_L03_<studentnumber>` exists before you log out.

Acceptance checklist

- ☐ The build is clean. Every declaration matches its definition.
- ☐ Lab 3 results equal the Lab 2 results (Checkpoint B table), and the Lab 2 project is unchanged.
- ☐ `led_init`, `led_on`, `led_off` use only register writes; there is no `HAL_GPIO_...` call in your code.
- ☐ The register table of Checkpoint C is filled in.
- ☐ LD2 blinks with `led_on` / `led_off` (Checkpoint D).
- ☐ All `main.c` edits are inside the `USER CODE` regions.

Individual oral check

Be ready for **one** question **or** one live change, chosen by your lab assistant. For a live change, predict the result first, then make the change.

Questions:

- What is the difference between a declaration and a definition? Where is each one for `led_on`?
- Why does `average_samples` need the count as a parameter?
- Why does `led_init` use `|=` for the clock, but `led_on` uses `=` for `BSRR`?
- What does `3U << 10` select, and why does PA5 start at bit 10?

Live changes:

- Change both blink delays to `100` ms. Do the masks change?
- Change the HIGH limit from `3000` to `2000`. How many blinks now?
- Replace the literal `5` with `#define LED_PIN 5U` in `led_on` and `led_off`.

Submit

Submit the four module files, `CMakeLists.txt`, the `main.c` user-code sections, the Checkpoint B and C tables, and one photo or short video of LD2 blinking.

Marks (10): pre-lab quiz on CATS 1 · A new project and build 1 · B functions 2 · C `led_init` and registers 2 · D `led_on`/`led_off` 2 · oral check (one question or one live change) 2. Checkpoint E is optional and not graded.